

CS 4476 PS4

Devin Fromond

dfromond3@gatech.edu

903761713

Part 1: Standard Scaler: Why did we use StandardScaler instead of looping over all the dataset twice for mean and standard deviation? Why a simple loop will not be a good choice in a deployed production grade ML system?

<text answer here>

We used StandardScaler since it is able to compute both the mean and standard deviation in one pass as opposed to computing these with slow loops. These loops do not really utilize numpy or any other additional speed-ups. With a large dataset, we require efficiency – especially over loops.

Part 1: Why do we normalize our data (0 mean, unit standard deviation)?

<text answer here>

We must normalize our data to ensure all features have the same scale and so no one feature dominates others. This goes back to the fairness principle and ensuring the model does not have any inherent bias.

Part 3: Loss function. Why did we need a loss function?

<text answer here>

We need a loss function to determine how far the model's predictions are from the actual values of the data. The loss function provides feedback on its errors to dynamically improve accuracy during training.

Part 3: [2 pt] KL-divergence is defined as

$$D_{KL}(P||Q) = \mathbb{E}_{X \sim P} \left[\log \frac{P(x)}{Q(x)} \right]$$

Cross-entropy is defined as

$$H(P, Q) = -\mathbb{E}_{X \sim P} \log Q(x)$$

How would you write cross-entropy in terms of KL-divergence

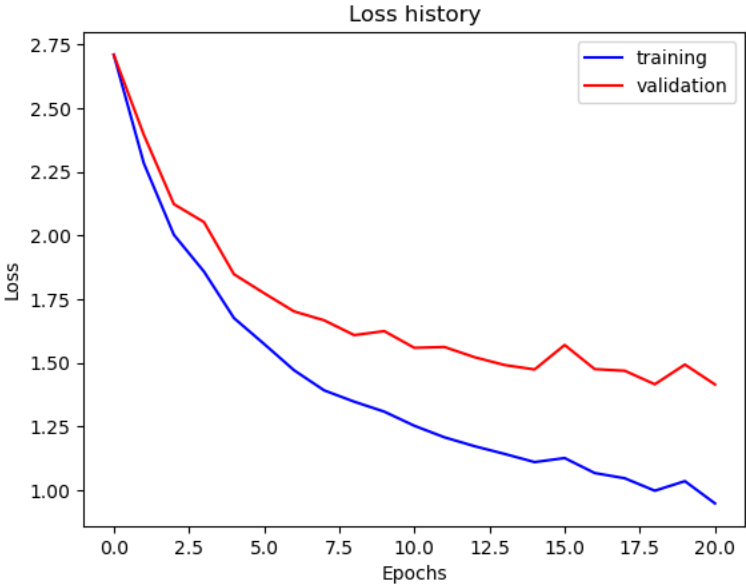
<text answer here>

We can define cross-entropy as the following:

$$H(P, Q) = D_{KL}(P||Q) + H(P)$$

Part 5: Training SimpleNet

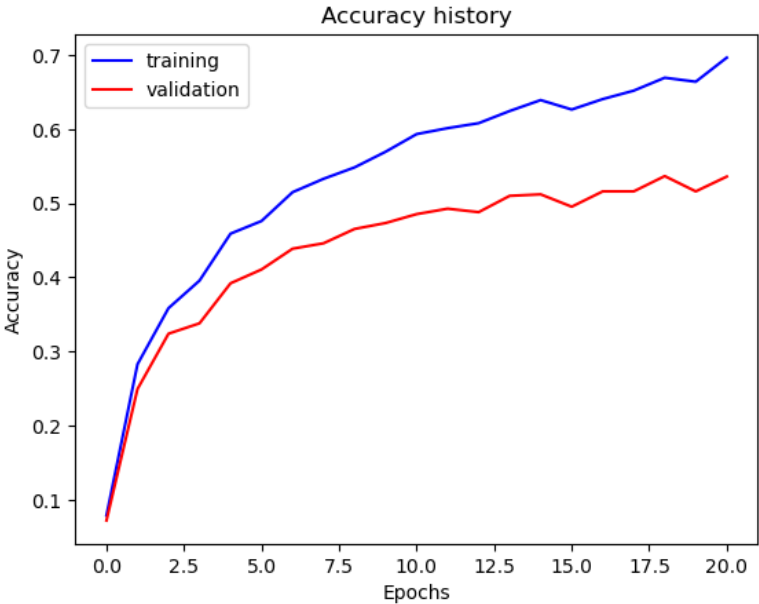
<Loss plot here>



Final training accuracy value: 0.6965

Final validation accuracy value: 0.5360

<Accuracy plot here>



Part 6: Simple Segmentation Net

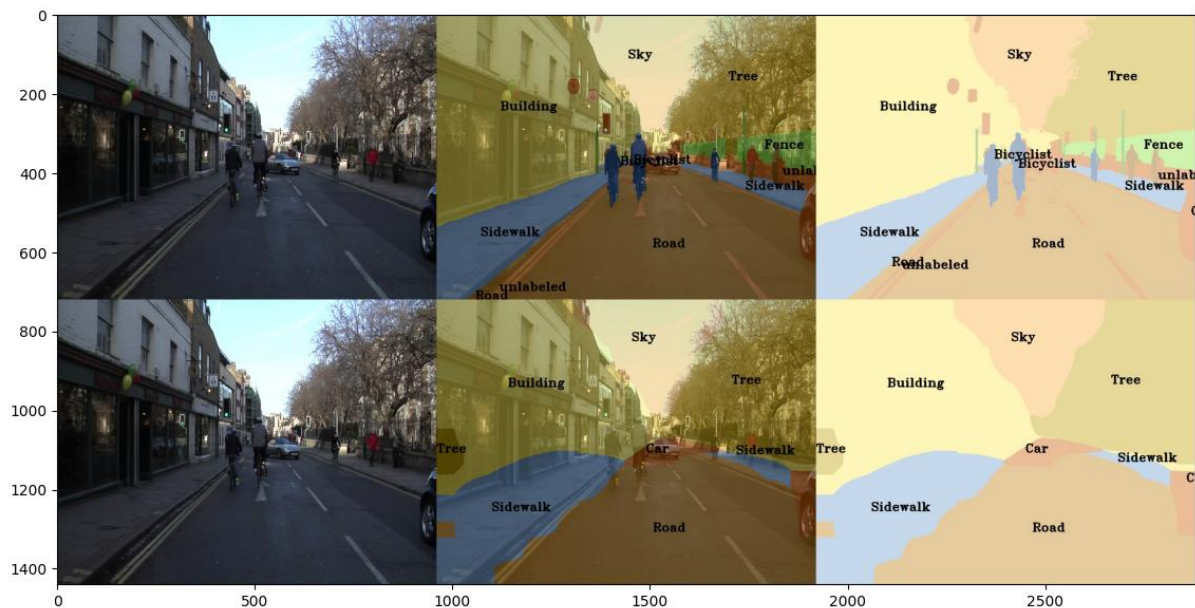
Class Index	Class name	Simple Segmentation Net Class IoU
0	Building	0.7874
1	Tree	0.7555
2	Sky	0.8431
3	Car	0.5973
4	SignSymbol	0.0000
5	Road	0.8502
6	Pedestrian	0.0000
7	Fence	0.0000
8	Column_Pole	0.0000
9	Sidewalk	0.5608
10	Bicyclist	0.0000

Validation mIoU: 0.4013

Number of Epochs: 5

Part 6: Simple Segmentation Net

Paste a figure of the generated semantic segmentation from Colab. It should be a 2x3 grid, with ground truth on the top row, and your predictions on the bottom row.



Part 6: Simple Segmentation Net

Which classes have the lowest mIoU? Why might they be the most difficult? Provide an example RGB image from Camvid that illustrates your point.



Classes that tend to contain smaller objects or more granular detail are more difficult to detect. Using the image to the left as reference, we can see bicyclists were not identified as well as more granular details on the right side – such as a fence.

Conclusion: briefly discuss what you have learned from this project.

<Text solution here>

This project has pulled back the curtain on image classification machine algorithms from being a black box to learning how to tune hyperparameters and what the strengths of the models are.

Theory: Neural Nets

1. Given are the layers of a neural network.
Calculate the total number of learnable parameters for each layer and in total.

- a) Conv 1: $(3 \times 5 \times 5)(64) + 64 = 4864$
- b) Conv 2: $(64 \times 3 \times 3)(128) + 128 = 73856$
- c) FC1: $(32100 \times 10) + 10 = 32010$
- d) Total (sum all 3): 110730

```
(conv1): Conv2d(3, 64, kernel_size=(5, 5), stride=(1, 1))
(maxpool1): MaxPool2d(kernel_size=2, stride=2, padding=0)
(conv2): Conv2d(64, 128, kernel_size=(3, 3), stride=(1, 1))
(maxpool2): MaxPool2d(kernel_size=2, stride=2, padding=0)
(fc1): Linear(in_features=3200, out_features=10, bias=True)
(softmax): Softmax(dim=1)
```

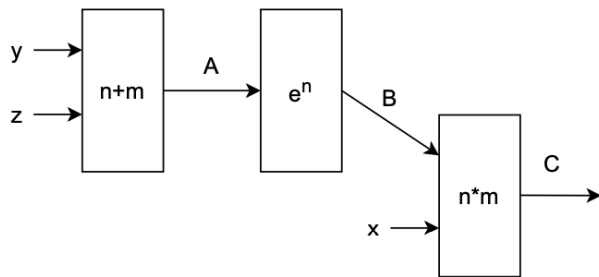
Note: You can look to [pytorch docs](https://pytorch.org/docs/) to understand what each of the parameters correspond to.

Theory: Neural Nets (cont.)

Here is the computational graph for the function $f(x, y, z) = x * e^{y+z}$. We want to compute the partial derivative of this function with respect to each input. Here are the three intermediate functions

$$A = y + z \quad B = e^A \quad C = x * B$$

From these we build the feed forward computation graph, with inputs n, m . The top input is n and the bottom is m .



a. What is the correct expression for $\frac{\partial C}{\partial x}$?

$$e^{y+z} = B$$

b. What is the correct expression for $\frac{\partial C}{\partial B}$?

$$x$$

c. What is the correct expression for $\frac{\partial B}{\partial A}$?

$$e^A = B$$

d. What is the correct expression for $\frac{\partial A}{\partial y}$?

$$1$$

e. What is the correct expression for $\frac{\partial C}{\partial y}$?

$$xe^{y+z} = xB$$

EC1.1: Screenshot of your get_data_augmentation_transforms()

<Screenshot here if attempted; do not delete the slide if not attempted>

```
def get_data_augmentation_transforms(
    inp_size: Tuple[int, int], pixel_mean: np.array, pixel_std: np.array
) -> transforms.Compose:
    """
    Returns the data augmentation + core transforms needed to be applied on the train set. Put data augmentation transforms before core transforms.

    Note: You can use transforms directly from torchvision.transforms

    Suggestions: Jittering, Flipping, Cropping, Rotating.

    Args:
    - inp_size: tuple denoting the dimensions for input to the model
    - pixel_mean: the mean of the raw dataset
    - pixel_std: the standard deviation of the raw dataset
    Returns:
    - aug_transforms: transforms.compose with all the transforms
    """

    #comment out the raise error when you start writing code
    # raise NotImplementedError('get_data_augmentation_transforms not implemented')

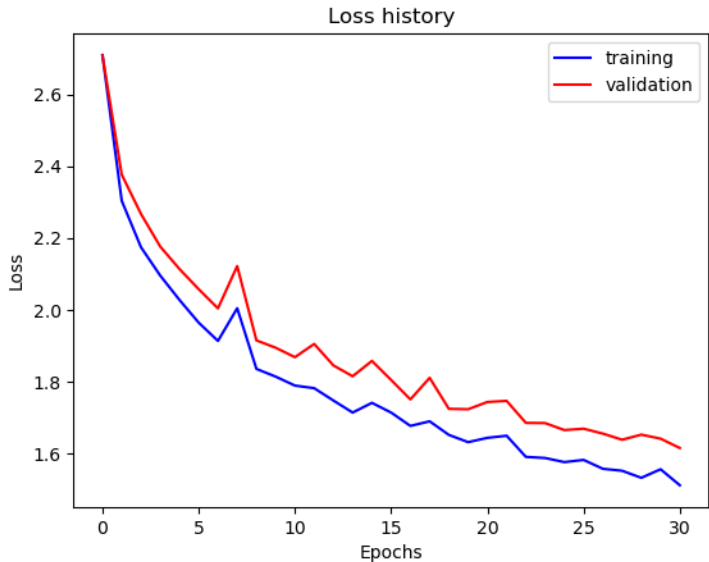
    return transforms.Compose(
        [
            #####
            # Student code begin
            #####

            transforms.ColorJitter(
                brightness=0.2, contrast=0.2, saturation=0.2, hue=0.1
            ),
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.RandomRotation(degrees=15),
            transforms.RandomResizedCrop(inp_size, scale=(0.8, 1.0)),
            transforms.ToTensor(),
            transforms.Normalize(mean=pixel_mean, std=pixel_std),

            #####
            # Student code end
            #####
        ]
    )
```

EC1: Training to solve overfitting

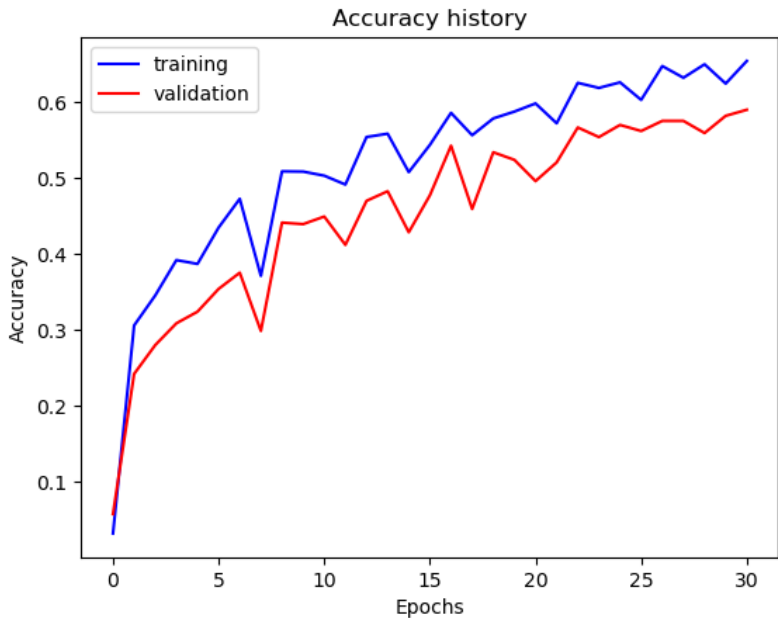
<Loss plot here>



Final training accuracy value: 0.6543

Final validation accuracy value: 0.5900

<Accuracy plot here>



EC2: PSPNet

Class Index	Class name	PSPNet Class IoU
0	Building	0. 8942
1	Tree	0. 8552
2	Sky	0. 9148
3	Car	0. 8551
4	SignSymbol	0. 0000
5	Road	0. 9302
6	Pedestrian	0. 1859
7	Fence	0. 5235
8	Column_Pole	0. 0010
9	Sidewalk	0. 8205
10	Bicyclist	0. 5842

Validation mIoU: 0.6124

Number of Epochs: 55

EC2: PSPNet

Paste a figure of the generated semantic segmentation from Colab. It should be a 2x3 grid, with ground truth on the top row, and your predictions on the bottom row.

